

Name: Syed Sheharyar Tariq
Roll No: 25F-AI-077
Subject: OOPS Manual
Instructor: Mam Noor-ul-Huda

Lab Manual 01

1. Create a date class. Its member data should consist of three integers: day, month, and year. It should also have three member functions: `getdate()` which allows the user to enter a date, `showdate()` which displays the date and `changeformat()` which change the date in mm/dd/yyyy format.

Code:

```
#include <iostream>
using namespace std;

class date {
    int day, month, year;
public:
    void getdate() {
        cin >> day >> month >> year;
    }
    void showdate() {
        cout << day << "/" << month << "/" << year << endl;
    }
    void changeformat() {
        cout << month << "/" << day << "/" << year << endl;
    }
};

int main() {
    date d;
    d.getdate();
    d.showdate();
    d.changeformat();
    return 0;
}
```

Output:

Compile Result

```
14
06
2006
14/6/2006
In Change Format: 6/14/2006

[Process completed - press Enter]
```

2. Create simple calculator using OOP

Code:

```
#include <iostream>
using namespace std;

class Calculator {
    double num1;
    double num2;
public:
    void setNumbers(double n1, double n2) {
        num1 = n1;
        num2 = n2;
    }

    double add() {
        return num1 + num2;
    }

    double subtract() {
        return num1 - num2;
    }

    double multiply() {
        return num1 * num2;
    }
}
```

```

    }

    double divide() {
        if(num2 != 0) {
            return num1 / num2;
        }
        return 0;
    }
};

int main() {
    Calculator calc;
    double a, b;
    char op;

    cin >> a >> op >> b;
    calc.setNumbers(a, b);

    if(op == '+') cout << calc.add() << endl;
    else if(op == '-') cout << calc.subtract() << endl;
    else if(op == '*') cout << calc.multiply() << endl;
    else if(op == '/') cout << calc.divide() << endl;

    return 0;
}

```

Output:

Compile Result

```

40
/
5
8

```

```

[Process completed - press Enter]

```

3. Write a c++ main function to create objects of distance class then input two distances and output the sum.

Detail:

Develop a class to measure distance as feet (should be int), inches (should be float). Include member functions to set and get attributes. Include constructors. Develop functions to add two distances.

Code:

```
#include <iostream>
using namespace std;

class Distance {
    int feet;
    float inches;
public:
    Distance() : feet(0), inches(0.0) {}
    Distance(int f, float i) : feet(f), inches(i) {}

    void setDistance(int f, float i) {
        feet = f;
        inches = i;
    }
    void getDistance() {
        cin >> feet >> inches;
    }
    Distance addDistance(Distance d2) {
        Distance sum;
        sum.inches = inches + d2.inches;
        sum.feet = feet + d2.feet + (int)(sum.inches / 12);
        sum.inches -= (int)(sum.inches / 12) * 12;
        return sum;
    }
    void show() {
        cout << feet << " feet " << inches << " inches" << endl;
    }
};

int main() {
    Distance d1, d2, d3;
    d1.getDistance();
    d2.getDistance();
    d3 = d1.addDistance(d2);
```

```

d3.show();
return 0;
}

```

Output:

```

Compile Result

Enter Feet then Inches: 5
19
Enter Feet then Inches: 7
22
15 feet 5 inches

[Process completed - press Enter]

```

4. Write a Program to Implement a Class STUDENT having following members:

Data members	
Member	Description
sname	Name of the student
Marks array	Marks of the student
total	Total marks obtained
Tmax	Total maximum marks

Member functions	
Member	Description
assign()	Assign Initial Values
compute()	to Compute Total, Average
display()	to Display the Data.

Code:

```

#include <iostream>
#include <string>

```

```

using namespace std;

class STUDENT {
    string sname;
    float Marks[5];
    float total;
    float Tmax;
public:
    void assign() {
        cin >> sname;
        for(int i = 0; i < 5; i++) {
            cin >> Marks[i];
        }
        cin >> Tmax;
    }
    void compute() {
        total = 0;
        for(int i = 0; i < 5; i++) {
            total += Marks[i];
        }
    }
    void display() {
        cout << sname << " Total: " << total << "/" << Tmax << " Avg: " << total / 5.0 << endl;
    }
};

int main() {
    STUDENT s;
    s.assign();
    s.compute();
    s.display();
    return 0;
}

```

Output:

Compile Result

Sheharyar

77

89

45

65

95

500

Sheharyar Total: 371/500 Avg: 74.2

[Process completed - press Enter]

Name: Syed Sheharyar Tariq
Roll No: 25F-AI-077
Subject: Oops Manual
Instructor: Mam Noor-ul-Huda

Lab Manual 02

1. Write a Program using copy constructor to copy data of an object to another object.

Code:

```
#include <iostream>
using namespace std;

class Box {
    int size;
public:
    Box(int s) : size(s) {}
    Box(const Box &obj) {
        size = obj.size;
    }
    void show() {
        cout << size << endl;
    }
};

int main() {
    Box b1(20);
    Box b2 = b1;
    b1.show();
    b2.show();
    return 0;
}
```

Output:

Compile Result

20

20

[Process completed - press Enter]

2. Write a Program to create 3 objects using constructor and destroy the same 3 objects using destructor.

Code:

```
#include <iostream>
using namespace std;

class Item {
    int id;
public:
    Item(int i) : id(i) {
        cout << id << endl;
    }
    ~Item() {
        cout << id << endl;
    }
};

int main() {
    Item item1(1);
    Item item2(2);
    Item item3(3);
    return 0;
}
```

Output:

Compile Result

Created with ID: 1

Created with ID: 2

Created with ID: 3

Destroyed ID: 3

Destroyed ID: 2

Destroyed ID: 1

[Process completed - press Enter]

Name: Syed Sheharyar Tariq
Roll No: 25F-AI-077
Subject: OOPS Manual
Instructor: Mam Noor-ul-Huda

Lab Manual 03

1. Create a class called employee that contains a name (string), an employee number (long) and employee joining date (struct date). Include a member function called getdata() to get data from the user for insertion into the object, and another function called displaydata() to display the data.
2. Write a main program to exercise this class. It should create 5 objects of class employee and then invite the user to input data. Finally it should print out data for all employees.

Code:

```
#include <iostream>
#include <string>
using namespace std;

struct date {
    int day, month, year;
};

class employee {
    string name;
    long emp_num;
    date joining_date;
public:
    void getdata() {
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Employee Number: ";
        cin >> emp_num;
        cout << "Enter joining date (Day Month Year): ";
        cin >> joining_date.day >> joining_date.month >> joining_date.year;
    }
    void displaydata() {
        cout << "Name: " << name << " | Number: " << emp_num << " | Joined: " << joining_date.day <<
        "/" << joining_date.month << "/" << joining_date.year << endl;
    }
};
```

```
    }  
};  
  
int main() {  
    employee emp[5];  
    for(int i = 0; i < 5; i++) {  
        cout << "Input for Employee " << i + 1 << ":" << endl;  
        emp[i].getdata();  
    }  
    cout << "\nDisplaying All Employee Data:" << endl;  
    for(int i = 0; i < 5; i++) {  
        emp[i].displaydata();  
    }  
    return 0;  
}
```

Output:

5:01



LTE



30%

Compile Result

```
Input for Employee 1:
Enter Name: Sheharyar
Enter Employee Number: 1
Enter joining date (Day Month Year): 14
06
2006
Input for Employee 2:
Enter Name: Saad
Enter Employee Number: 2
Enter joining date (Day Month Year): 07
05
2007
Input for Employee 3:
Enter Name: Haya
Enter Employee Number: 3
Enter joining date (Day Month Year): 08
01
2016
Input for Employee 4:
Enter Name: Raza
Enter Employee Number: 4
Enter joining date (Day Month Year): 25
11
2000
Input for Employee 5:
Enter Name: Saif
Enter Employee Number: 5
Enter joining date (Day Month Year): 29
```

3. Create a class Matrix_Math of following type.

```
Class Matrix_Math
{
    Private:
    int subject_marks[5];

    Public:
    void max_marks();
    void min_marks();
    void average_marks();
}
```

Code:

```
#include <iostream>
using namespace std;

class Matrix_Math {
private:
    int subject_marks[5];
public:
    void get_marks() {
        for(int i = 0; i < 5; i++) {
            cin >> subject_marks[i];
        }
    }
    void max_marks() {
        int max = subject_marks[0];
        for(int i = 1; i < 5; i++) {
            if(subject_marks[i] > max) max = subject_marks[i];
        }
        cout << max << endl;
    }
    void min_marks() {
        int min = subject_marks[0];
        for(int i = 1; i < 5; i++) {
            if(subject_marks[i] < min) min = subject_marks[i];
        }
        cout << min << endl;
    }
    void average_marks() {
        int sum = 0;
```

```
        for(int i = 0; i < 5; i++) {
            sum += subject_marks[i];
        }
        cout << sum / 5.0 << endl;
    }
};

int main() {
    Matrix_Math m;
    m.get_marks();
    m.max_marks();
    m.min_marks();
    m.average_marks();
    return 0;
}
```

Output:

Compile Result

56

90

45

78

77

Maximum: 90

Minimum: 45

Average: 69.2

[Process completed - press Enter]

Name: Syed Sheharyar Tariq
Roll No: 25F-AI-077
Subject: OOps Manual
Instructor: Mam Noor-ul-Huda

Lab Manual 04

1. Write down the basic program using inheritance concept; at least using more than two child class derived from basic class.

Code:

```
#include <iostream>
using namespace std;

class Polygon {
protected:
    double width, height;
public:
    void set_values(double a, double b) {
        width = a; height = b;
    }
};

class Rectangle : public Polygon {
public:
    double area() { return width * height; }
};

class Triangle : public Polygon {
public:
    double area() { return (width * height) / 2; }
};

int main() {
    Rectangle rect;
    Triangle tri;
    rect.set_values(4, 5);
    tri.set_values(4, 5);
    cout << rect.area() << endl;
    cout << tri.area() << endl;
    return 0;
}
```


Output:

```
Compile Result

20
10

[Process completed - press Enter]
```

2. Create a class named Trigon inherited from the class GeometricShape (containing a single data member named shapeName (a string) and a member function named setShapeName(shapeName). The class Trigon is required to hold:

Three data members i.e. base, perpendicular, and hypotenuse of type double

A no-arg constructor that initializes all data members with value 1.0

A parameterized constructor to initialize all data fields with user-defined values

The setter functions for all three data fields

The accessor functions for all three data fields

A function named displayArea() that shows the area of a certain Trigon object

In the main() function make three objects of class Trigon while considering the shapeName as “threeangle” for each object. In addition, it is required to invoke all the functions of Trigon class.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class GeometricShape {
protected:
    string shapeName;
public:
    void setShapeName(string name) {
        shapeName = name;
```

```

    }
};

class Trigon : public GeometricShape {
    double base, perpendicular, hypotenuse;
public:
    Trigon() : base(1.0), perpendicular(1.0), hypotenuse(1.0) {}
    Trigon(double b, double p, double h) : base(b), perpendicular(p), hypotenuse(h) {}

    void setBase(double b) { base = b; }
    void setPerpendicular(double p) { perpendicular = p; }
    void setHypotenuse(double h) { hypotenuse = h; }

    double getBase() { return base; }
    double getPerpendicular() { return perpendicular; }
    double getHypotenuse() { return hypotenuse; }

    void displayArea() {
        cout << shapeName << " Area: " << 0.5 * base * perpendicular << endl;
    }
};

int main() {
    Trigon t1;
    t1.setShapeName("threeangle");
    t1.displayArea();

    Trigon t2(4.0, 5.0, 6.0);
    t2.setShapeName("threeangle");
    t2.displayArea();

    Trigon t3;
    t3.setShapeName("threeangle");
    t3.setBase(10.0);
    t3.setPerpendicular(5.0);
    t3.setHypotenuse(12.0);
    t3.displayArea();

    return 0;
}

```

Output:

Compile Result

```
threeangle t1 Area: 0.5  
threeangle t2 Area: 10  
threeangle t3 Area: 25
```

```
[Process completed - press Enter]
```

3. Consider a base class named **Employee** and its derived classes **HourlyEmployee** and **PermanentEmployee** while taking into account the following criteria.

Employee class has two data fields i.e. a name (of type string) and specific empID (of type integer)

Both classes (**HourlyEmployee** and **PermanentEmployee**) have an attribute named **hourlyIncome**

Both classes (**HourlyEmployee** and **PermanentEmployee**) have three-argument constructor to initialize the **hourlyIncome** as well as data fields of the base class

Class **HourlyEmployee** has a function named **calculate_the_hourly_income** to calculate the income of an employee for the actual number of hours he or she worked. One hour income is Rs. 150

Similarly, **PermanentEmployee** class has function named **calculate_the_income** to calculate the income of an employee that gets paid the salary for exact 240 hours, no matter how many actual hours he or she worked. Again, one hour salary is Rs. 150.

Code:

```
#include <iostream>  
#include <string>  
using namespace std;  
  
class Employee {  
protected:  
    string name;  
    int empID;  
public:  
    Employee(string n, int id) : name(n), empID(id) {}  
};
```

```

};

class HourlyEmployee : public Employee {
    double hourlyIncome;
public:
    HourlyEmployee(string n, int id, double hi) : Employee(n, id), hourlyIncome(hi) {}
    void calculate_the_hourly_income(int hours) {
        cout << name << " Income: " << hours * 150 << endl;
    }
};

class PermanentEmployee : public Employee {
    double hourlyIncome;
public:
    PermanentEmployee(string n, int id, double hi) : Employee(n, id), hourlyIncome(hi) {}
    void calculate_the_income() {
        cout << name << " Income: " << 240 * 150 << endl;
    }
};

int main() {
    HourlyEmployee he("Ali", 1001, 150.0);
    he.calculate_the_hourly_income(100);

    PermanentEmployee pe("Sara", 1002, 150.0);
    pe.calculate_the_income();

    return 0;
}

```

Output:

Compile Result

```
Ali Income: 15000  
Sara Income: 36000
```

```
[Process completed - press Enter]
```

Name: Syed Sheharyar Tariq
Roll No: 25F-AI-077
Subject: Oops Manual
Instructor: Mam Noor-ul-Huda

Lab Manual 05

1. Write down a program using multiple inheritance, using following scenario: Person, Employee -> Teacher.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;
public:
    void getPerson() {
        cout << "Enter name: ";
        cin >> name;
    }
};

class Employee {
protected:
    int empId;
public:
    void getEmployee() {
        cout << "Enter Employee ID: ";
        cin >> empId;
    }
};

class Teacher : public Person, public Employee {
public:
    void display() {
        cout << "Name: " << name << "\nID: " << empId << endl;
    }
};
```

```
int main() {  
    Teacher t;  
    t.getPerson();  
    t.getEmployee();  
    t.display();  
    return 0;  
}
```

Output:

Compile Result

```
Enter name: Sheharyar  
Enter Employee ID: 1101  
Name: Sheharyar  
ID: 1101
```

```
[Process completed - press Enter]
```

2. Consider four classes named Animal, Mammal, Bird, and Bat. Class Animal includes the functionality of eating and breathing. Class Mammal is derived from Animal class and has additional functionality of giving birth but not hatching. Class Bird is derived from Animal class and has specific functionality of flying. Class Bat is derived from Mammal and Bird classes while inheriting the functionality of both these classes. Illustrate the implementation of these classes in C++.

Code:

```
#include <iostream>  
using namespace std;  
  
class Animal {  
public:  
    void eat() { cout << "Eating..." << endl; }  
    void breathe() { cout << "Breathing..." << endl; }  
};
```

```
class Mammal : virtual public Animal {
public:
    void giveBirth() { cout << "Giving birth..." << endl; }
};

class Bird : virtual public Animal {
public:
    void fly() { cout << "Flying..." << endl; }
};

class Bat : public Mammal, public Bird {
public:
    void display() {
        eat();
        breathe();
        giveBirth();
        fly();
    }
};

int main() {
    Bat b;
    b.display();
    return 0;
}
```

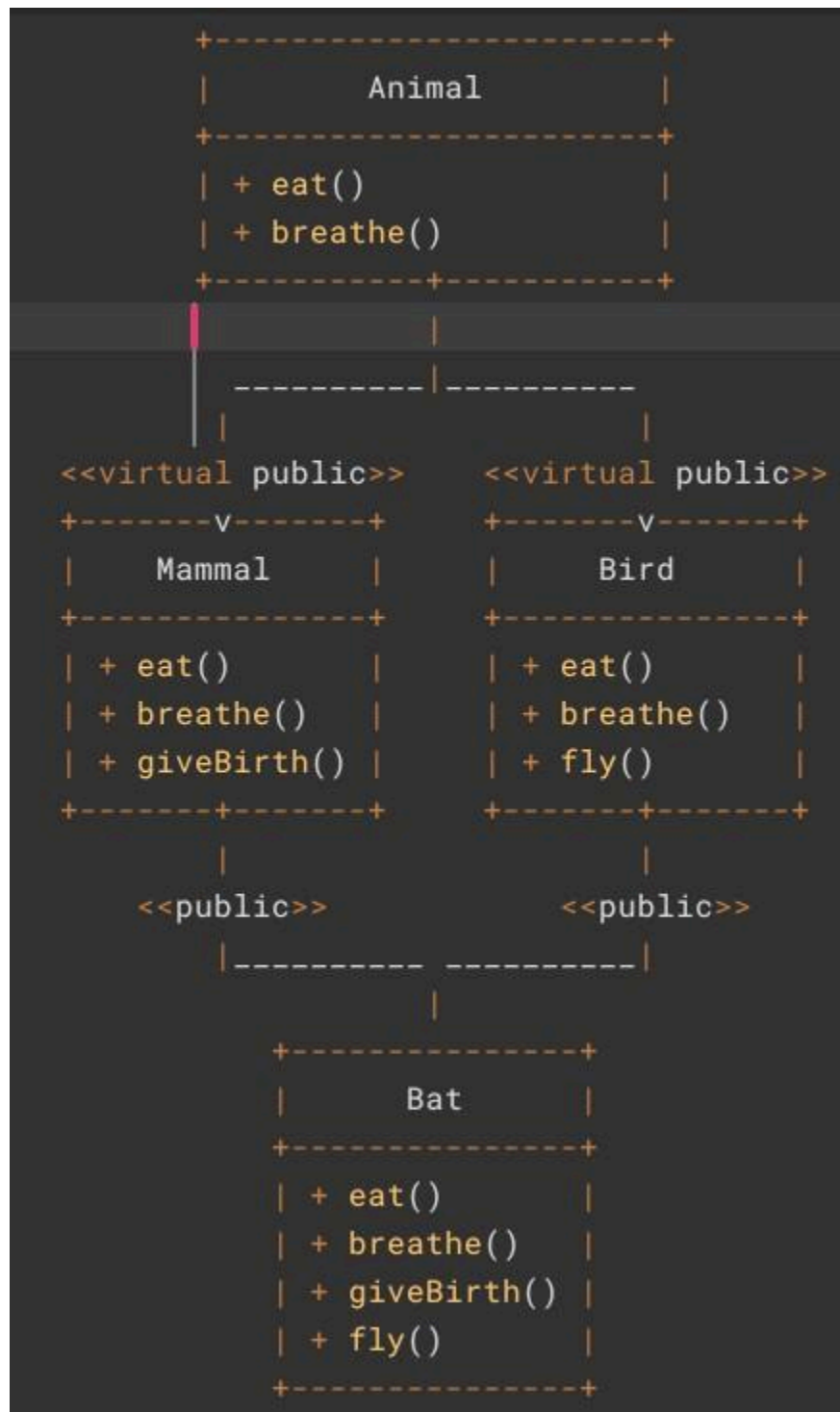
Output:

Compile Result

```
Eating...
Breathing...
Giving birth...
Flying...
```

```
[Process completed - press Enter]
```


UML Diagram:



3. Define a class **Student** as a base class with data members as `admissionNo`, `Name`, `age` and `address` as protected type and `getStudent` function for setting values. Make two

derived classes: Undergraduate and Graduate each containing data member degree_program and member functions getdegree and show.

Code:

```
#include <iostream>
#include <string>
using namespace std;

class Student {
protected:
    int admissionNo;
    string name;
    int age;
    string address;
public:
    void getStudent() {
        cout << "Admission No: "; cin >> admissionNo;
        cout << "Name: "; cin >> name;
        cout << "Age: "; cin >> age;
        cout << "Address: "; cin >> address;
    }
};

class Undergraduate : public Student {
    string degree_program;
public:
    void getdegree() {
        cout << "Degree Program: "; cin >> degree_program;
    }
    void show() {
        cout << "\n** Undergraduate Student **\n";
        cout << "Admission No: " << admissionNo << "\nStudent Name: " << name << "\nAge: " << age <<
"\nDegree Program: " << degree_program << endl;
    }
};

class Graduate : public Student {
    string degree_program;
public:
    void getdegree() {
        cout << "Degree Program: "; cin >> degree_program;
    }
    void show() {
```

```
        cout << "\n** Graduate Student **\n";
        cout << "Admission No: " << admissionNo << "\nStudent Name: " << name << "\nAge: " << age <<
"\nDegree Program: " << degree_program << endl;
    }
};
```

```
int main() {
    Graduate g;
    g.getStudent();
    g.getdegree();
    g.show();

    Undergraduate u;
    u.getStudent();
    u.getdegree();
    u.show();
    return 0;
}
```

Output:

Compile Result

Admission No: 77
Name: Sheharyar
Age: 19
Address: Maymar
Degree Program: AI

**** Graduate Student ****
Admission No: 77
Student Name: Sheharyar
Age: 19
Degree Program: AI

Admission No: 77
Name: Sheharyar
Age: 19
Address: Maymar
Degree Program: AI

**** Undergraduate Student ****
Admission No: 77
Student Name: Sheharyar
Age: 19
Degree Program: AI

[Process completed - press Enter]

4. By using multiple inheritance, make a student class representing student with data member roll_no. A test class (derived class of student) representing scores in two subjects and sports class (derived class of student) representing score in sports. The sports and test class should be inherited by a result class to add scores and display the final result.

Code:

```
#include <iostream>
using namespace std;

class Student {
protected:
    int roll_no;
public:
    void get_roll_no() {
        cout << "Enter Roll No: "; cin >> roll_no;
    }
    void display_roll_no() {
        cout << "Roll No: " << roll_no << endl;
    }
};

class Test : virtual public Student {
protected:
    float s1, s2;
public:
    void get_test() {
        cout << "Enter marks for two subjects: "; cin >> s1 >> s2;
    }
};

class Sports : virtual public Student {
protected:
    float score;
public:
    void get_sports() {
        cout << "Enter sports score: "; cin >> score;
    }
};

class Result : public Test, public Sports {
    float total;
```

```
public:
    void display() {
        total = s1 + s2 + score;
        display_roll_no();
        cout << "Total Result: " << total << endl;
    }
};

int main() {
    Result res;
    res.get_roll_no();
    res.get_test();
    res.get_sports();
    res.display();
    return 0;
}
```

Output:

Compile Result

```
Enter Roll No: 77
Enter marks for two subjects: 90
88
Enter sports score: 100
Roll No: 77
Total Result: 278

[Process completed - press Enter]
```

Name: Syed Sheharyar Tariq
Roll No: 25F-AI-077
Subject: Oops Manual
Instructor: Mam Noor-ul-Huda

Lab Manual 06

1. Write a Distance class that contains private variables inches, feet. Declare a friend function named DistSqr() that takes a distance value as an argument and returns the square of the distance (convert to feet then square).

Code:

```
#include <iostream>
using namespace std;

class Distance {
    float feet, inches;
public:
    Distance(float f, float i) : feet(f), inches(i) {}
    friend float DistSqr(Distance);
};

float DistSqr(Distance d) {
    float totalFeet = d.feet + (d.inches / 12.0);
    return totalFeet * totalFeet;
}

int main() {
    Distance d(5, 6);
    cout << "Square of distance in feet: " << DistSqr(d) << endl;
    return 0;
}
```

Output:

Compile Result

Square of distance in feet: 30.25

[Process completed - press Enter]

2. Adding members of two different classes using the friend function.

Code:

```
#include <iostream>
using namespace std;

class B;
class A {
    int valA;
public:
    A(int v) : valA(v) {}
    friend int add(A, B);
};

class B {
    int valB;
public:
    B(int v) : valB(v) {}
    friend int add(A, B);
};

int add(A a, B b) {
    return a.valA + b.valB;
}

int main() {
    A objA(10);
    B objB(20);
    cout << "Sum: " << add(objA, objB) << endl;
    return 0;
}
```


Output:

```
Compile Result

Sum: 30

[Process completed - press Enter]
```

3. Friend Function to Calculate Average.

Code:

```
#include <iostream>
using namespace std;

class Student {
    float m1, m2, m3;
public:
    Student(float a, float b, float c) : m1(a), m2(b), m3(c) {}
    friend float calculateAverage(Student);
};

float calculateAverage(Student s) {
    return (s.m1 + s.m2 + s.m3) / 3.0;
}

int main() {
    Student s(80, 90, 70);
    cout << "Average: " << calculateAverage(s) << endl;
    return 0;
}
```

Output:

Compile Result

Average: 80

[Process completed - press Enter]

4. Implement the following pseudo code.

Class A {

Data type: integer no1;

Function: friend function by the name SUM ().

}

Class B {

Data type: integer no2;

Function: friend function by the name SUM ().

}

SUM() should add the values no1, no2 from class A and B respectively.

Code:

```
#include <iostream>
using namespace std;
```

```
class B;
class A {
    int no1;
public:
    A(int n) : no1(n) {}
```

```
    friend void SUM(A, B);
};

class B {
    int no2;
public:
    B(int n) : no2(n) {}
    friend void SUM(A, B);
};

void SUM(A a, B b) {
    cout << "Sum: " << a.no1 + b.no2 << endl;
}

int main() {
    A objA(50);
    B objB(100);
    SUM(objA, objB);
    return 0;
}
```

Output:

Compile Result

Sum: 150

[Process completed - press Enter]